DAA – Java Programs

External Oral Examination – Code Explanation & Viva Notes

Five practicals covered

1. Fibonacci – Recursive and Iterative
2. Huffman Encoding
3. Fractional Knapsack
4. N-Queens Problem
5. Quick Sort Analysis – Deterministic and Randomized

Short explanations of important Java code, algorithm logic, complexity, viva questions and key differences.
Based on the DAA programs in your uploaded A-40 PDF.

How to use these notes

Learn the aim, main logic, method purpose and time/space complexity. In the external, explain each program in execution order: input → algorithm → output. The notes follow the Java code in your practical PDF.

The five DAA practicals are Fibonacci (two approaches), Huffman Encoding, Fractional Knapsack, N-Queens and Quick Sort Analysis.

1. Fibonacci – Recursive and Iterative

PDF pages 93–94 | Aim: Find the nth Fibonacci number and compare recursive and iterative approaches.

A. Recursive Fibonacci

```
import java.util.Scanner;
public class FibonacciRecursive {
static int stepCount = 0;
static int fibonacci(int n) {
stepCount++;
if (n == 0) return 0;
if (n == 1) return 1;
return fibonacci(n - 1) + fibonacci(n - 2);
}
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int result = fibonacci(n);
System.out.println("Fibonacci number = " + result);
System.out.println("Step Count = " + stepCount);
sc.close();
}
}
```

Code	Short explanation
import java.util.Scanner;	Imports Scanner for user input.
static int stepCount=0;	Global counter for recursive calls.
fibonacci(int n)	Method that returns the nth Fibonacci number.
stepCount++;	Counts each method call.
n==0 / n==1	Base cases: return 0 and 1.
fibonacci(n-1)+fibonacci(n-2)	Adds the previous two Fibonacci values.
new Scanner(System.in)	Creates input reader.
nextInt()	Reads an integer.
fibonacci(n)	Calls the recursive method.
println()	Displays result and call count.
sc.close()	Closes Scanner.

B. Iterative Fibonacci

```
int a = 0, b = 1, c;
int stepCount = 0;
if (n == 0) { ... }
else if (n == 1) { ... }
else {
for (int i = 2; i <= n; i++) {
c = a + b;
stepCount++;
a = b;
stepCount++;
b = c;
stepCount++;
}
System.out.println("Fibonacci number = " + b);
}
```

Code	Short explanation
a=0, b=1, c	Initializes first two values; c stores next value.
n==0 / n==1	Handles base cases directly.
for (i=2; i<=n; i++)	Repeats from third position through n.

Code	Short explanation
c=a+b	Calculates next Fibonacci number.
a=b	Moves second value into first position.
b=c	Stores new value as second value.
stepCount++	Counts the operations shown in this program.

Complexity: Recursive version is approximately O(2■) time and O(n) call-stack space. Iterative version is O(n) time and O(1) extra space.

2. Huffman Encoding

PDF pages 95–97 | Aim: Generate variable-length binary codes based on character frequencies.

A. Node class

```
class Node {
char ch;
int frequency;
Node left, right;
Node(char ch, int frequency) {
this.ch = ch;
this.frequency = frequency;
}
Node(int frequency, Node left, Node right) {
this.frequency = frequency;
this.left = left;
this.right = right;
}
}
```

Code	Short explanation
char ch	Stores a character.
int frequency	Stores how often the character occurs.
Node left, right	References left and right child nodes.
Node(char, frequency)	Constructor for a leaf node.
this.ch / this.frequency	Assigns constructor values to fields.
Node(frequency,left,right)	Constructor for an internal tree node.

B. Generating Huffman codes

```
if (root == null) return;
if (root.left == null && root.right == null) {
huffmanCode.put(root.ch, code);
return;
}
generateCodes(root.left, code + "0", huffmanCode);
generateCodes(root.right, code + "1", huffmanCode);
```

Code	Short explanation
root == null	Stops if current node does not exist.
left==null && right==null	Checks whether node is a leaf.
huffmanCode.put()	Stores character and its binary code.
code + "0"	Adds 0 when traversing left branch.
code + "1"	Adds 1 when traversing right branch.
generateCodes()	Recursively traverses the Huffman tree.

C. Frequency counting and tree building

```
String text = sc.nextLine();
Map<Character, Integer> frequency = new HashMap<>();
for (char ch : text.toCharArray()) {
frequency.put(ch, frequency.getOrDefault(ch, 0) + 1);
}
PriorityQueue<Node> pq = new PriorityQueue<>(
Comparator.comparingInt(node -> node.frequency)
);
for (Map.Entry<Character, Integer> entry : frequency.entrySet()) {
pq.add(new Node(entry.getKey(), entry.getValue()));
}
while (pq.size() > 1) {
Node left = pq.poll();
Node right = pq.poll();
Node newNode = new Node(left.frequency + right.frequency, left, right);
pq.add(newNode);
}
Node root = pq.poll();
```

Code	Short explanation
nextLine()	Reads the complete input text.
Map	Stores each character and its frequency.

Code	Short explanation
toCharArray()	Converts string into a character array.
getOrDefault(ch,0)+1	Gets existing count or zero, then increments it.
PriorityQueue	Priority queue of tree nodes.
Comparator.comparingInt()	Orders nodes by increasing frequency.
pq.add()	Inserts a node.
pq.poll()	Removes the smallest-frequency node.
while (pq.size()>1)	Combines nodes until one root remains.
new Node(sum,left,right)	Creates parent node with combined frequency.
root=pq.poll()	Gets final Huffman tree root.

D. Generate and encode

```
Map<Character, String> huffmanCode = new HashMap<>();
generateCodes(root, "", huffmanCode);
StringBuilder encodedText = new StringBuilder();
for (char ch : text.toCharArray()) {
encodedText.append(huffmanCode.get(ch));
}
```

HashMap stores character-to-code mappings. generateCodes() traverses the tree. StringBuilder builds the encoded output. Each input character is replaced with its Huffman code. Huffman coding is a lossless compression technique.

Complexity: Building the Huffman tree with a priority queue is typically O(n log n), where n is the number of distinct characters.

3. Fractional Knapsack Problem

PDF pages 99–100 | Aim: Maximize profit within a weight capacity, allowing fractions of items.

A. Item class

```
class Item {
int weight;
int profit;
double ratio;
Item(int weight, int profit) {
this.weight = weight;
this.profit = profit;
this.ratio = (double) profit / weight;
}
}
```

Code	Short explanation
weight / profit	Stores item's weight and profit.
double ratio	Stores profit per unit weight.
Item(...)	Constructor initializes item.
this.weight / this.profit	Assigns values to object fields.
(double) profit / weight	Calculates ratio using decimal division.

B. Input and item creation

```
int n = sc.nextInt();
Item[] items = new Item[n];
for (int i = 0; i < n; i++) {
int weight = sc.nextInt();
int profit = sc.nextInt();
items[i] = new Item(weight, profit);
}
int capacity = sc.nextInt();
```

Reads item count, creates an Item array, reads each weight and profit, creates Item objects, and reads knapsack capacity.

C. Greedy sorting and selection

```
Arrays.sort(items, (a, b) -> Double.compare(b.ratio, a.ratio));
double totalProfit = 0.0;
int remainingCapacity = capacity;
for (int i = 0; i < n; i++) {
if (remainingCapacity == 0) break;
if (items[i].weight <= remainingCapacity) {
remainingCapacity -= items[i].weight;
totalProfit += items[i].profit;
} else {
double fraction = (double) remainingCapacity / items[i].weight;
totalProfit += items[i].profit * fraction;
remainingCapacity = 0;
}
}
```

Code	Short explanation
Arrays.sort()	Sorts items.
Double.compare(b.ratio,a.ratio)	Sorts ratios in descending order.
totalProfit=0.0	Initializes total profit.
remainingCapacity=capacity	Stores unused capacity.
remainingCapacity==0	Stops when knapsack is full.
weight<=remainingCapacity	Checks whether complete item fits.
remainingCapacity -= weight	Reduces remaining capacity.
totalProfit += profit	Adds full item's profit.
fraction=remaining/weight	Calculates fraction of next item that fits.
profit*fraction	Adds proportional profit.
remainingCapacity=0	Marks knapsack full.

Greedy strategy: Choose the item with highest profit/weight ratio first. **Complexity:** O(n log n) for sorting; selection is O(n).

4. N-Queens Problem

PDF pages 101–102 | Aim: Place N queens on an N×N chessboard so no two queens attack each other.

A. Board and safety check

```
static int[][] board;
static int n;
static boolean isSafe(int row, int col) {
for (int i = 0; i < row; i++)
if (board[i][col] == 1) return false;
for (int i=row-1, j=col-1; i>=0 && j>=0; i--,j--)
if (board[i][j] == 1) return false;
for (int i=row-1, j=col+1; i>=0 && j<n; i--,j++)
if (board[i][j] == 1) return false;
return true;
}
```

Code	Short explanation
int[][] board	2D array representing chessboard.
int n	Stores board size.
isSafe(row,col)	Checks whether a queen can be placed.
column loop	Checks same column in previous rows.
board[i][col]==1	A queen exists there, so position is unsafe.
upper-left loop	Checks diagonal above and to the left.
upper-right loop	Checks diagonal above and to the right.
return true	No conflict found; position is safe.

B. Backtracking function

```
static boolean solveNQueens(int row) {
if (row == n) return true;
for (int col = 0; col < n; col++) {
if (isSafe(row, col)) {
board[row][col] = 1;
if (solveNQueens(row + 1)) return true;
board[row][col] = 0;
}
}
return false;
}
```

Code	Short explanation
solveNQueens(row)	Attempts to place queens starting from this row.
row==n	Base case: all queens placed.
for col=0 to n-1	Tries every column in current row.
isSafe()	Checks whether selected cell is valid.
board[row][col]=1	Places a queen.
solveNQueens(row+1)	Recursively solves next row.
board[row][col]=0	Backtracks by removing failed placement.
return false	No valid arrangement found from this state.

C. Display and main

```
static void displayBoard() {
for (int i = 0; i < n; i++) {
for (int j = 0; j < n; j++)
System.out.print(board[i][j] + " ");
System.out.println();
}
}
n = sc.nextInt();
board = new int[n][n];
board[0][0] = 1;
if (solveNQueens(1)) displayBoard();
```

displayBoard() prints the matrix row by row. Main reads N, creates an N×N board, places the first queen at row 0/column 0, and starts solving from row 1. Important: this particular code fixes the first queen at (1,1) in one-based notation; it does not search arrangements starting at other first-queen positions.

Technique: Backtracking tries a choice, explores recursively, and undoes it if it fails. **Complexity:** Exponential worst-case search; a commonly stated upper bound is O(N!).

5. Quick Sort Analysis – Deterministic and Randomized

PDF pages 103–105 | Aim: Sort an array using two pivot-selection approaches and compare comparisons and execution time.

A. Counters and deterministic Quick Sort

```
static long deterministicComparisons = 0;
static long randomizedComparisons = 0;
static void deterministicQuickSort(int[] arr, int low, int high) {
if (low < high) {
int p = deterministicPartition(arr, low, high);
deterministicQuickSort(arr, low, p - 1);
deterministicQuickSort(arr, p + 1, high);
}
}
```

Code	Short explanation
static long counters	Count comparisons for both versions.
deterministicQuickSort()	Recursively sorts left and right partitions.
low<high	Checks partition contains at least two elements.
deterministicPartition()	Partitions array and returns pivot index.
quickSort(low,p-1)	Sorts left side of pivot.
quickSort(p+1,high)	Sorts right side of pivot.

B. Deterministic partition

```
int pivot = arr[high];
int i = low - 1;
for (int j = low; j < high; j++) {
deterministicComparisons++;
if (arr[j] <= pivot) {
i++;
swap(arr, i, j);
}
}
swap(arr, i + 1, high);
return i + 1;
```

Code	Short explanation
pivot=arr[high]	Always chooses last element as pivot.
i=low-1	Tracks end of smaller/equal region.
j=low; j<high; j++	Scans elements before pivot.
comparisons++	Counts each comparison with pivot.
arr[j]<=pivot	Checks if current value belongs on left.
i++ and swap()	Moves smaller/equal value into left partition.
swap(i+1,high)	Places pivot in correct sorted position.
return i+1	Returns pivot index.

C. Randomized partition

```
Random rand = new Random();
int randomIndex = low + rand.nextInt(high - low + 1);
swap(arr, randomIndex, high);
int pivot = arr[high];
```

Code	Short explanation
new Random()	Creates random number generator.
nextInt(high-low+1)	Generates a value within partition size.
low + ...	Converts it to an array index in range.
swap(randomIndex,high)	Moves random pivot to the end.
pivot=arr[high]	Uses selected random pivot.
randomizedComparisons++	Counts randomized version comparisons.

After selecting the pivot, randomized partition uses the same scan, compare, swap and pivot-placement logic as deterministic partition.

D. Swap, display and timing

```
static void swap(int[] arr, int i, int j) {
int temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;
}
static void display(int[] arr) {
for (int x : arr) System.out.print(x + " ");
}
int[] deterministicArray = arr.clone();
int[] randomizedArray = arr.clone();
long start1 = System.nanoTime();
deterministicQuickSort(deterministicArray, 0, n - 1);
long end1 = System.nanoTime();
long start2 = System.nanoTime();
randomizedQuickSort(randomizedArray, 0, n - 1);
long end2 = System.nanoTime();
```

Code	Short explanation
swap()	Exchanges two elements using a temporary variable.
display()	Prints array elements.
arr.clone()	Creates separate copies for fair comparison.
System.nanoTime()	Reads high-resolution time.
start/end variables	Record start and end of each sorting run.
end-start	Calculates elapsed time in nanoseconds.

Main reads array size and elements, clones the input twice, runs both sorts, and prints original/sorted arrays, comparison counts and execution times.

Complexity: Best and average O(n log n); worst O(n^2). Random pivot selection lowers the chance of repeatedly unbalanced partitions but does not eliminate the worst case.

Important viva questions – all five programs

1. Fibonacci

Question	Short oral answer
What is Fibonacci series?	Each term is the sum of the previous two: 0, 1, 1, 2, 3, 5, ...
What is recursion?	A method calling itself to solve a smaller problem.
What are base cases?	Conditions that stop recursion; here n=0 and n=1.
What is iteration?	Repeating instructions using loops.
Which Fibonacci approach is faster?	Iterative; it avoids repeated recursive calculations.
Recursive time complexity?	Approximately O(2^n) for this simple implementation.
Iterative time complexity?	O(n).
What is stepCount?	A counter for recursive calls or operations, as coded.

2. Huffman Encoding

Question	Short oral answer
What is Huffman coding?	A lossless compression technique using variable-length binary codes.
What is a greedy algorithm?	It makes the locally best choice at each step.
Why use a priority queue?	To remove the two nodes with smallest frequencies efficiently.
What is a leaf node?	A node with no children.
Why add 0 left and 1 right?	To form binary paths used as character codes.
What is frequency?	Number of times a character appears.
Is Huffman coding lossless?	Yes; the original data can be decoded using the Huffman tree.

3. Fractional Knapsack

Question	Short oral answer
What is Knapsack problem?	Select items to maximize profit within a weight capacity.
What is Fractional Knapsack?	A version where items may be taken partially.
Which strategy is used?	Greedy strategy.
What is profit/weight ratio?	Profit earned per unit of weight.
Why sort descending?	To choose highest profit-per-weight items first.
Can an item be divided?	Yes, in Fractional Knapsack.
Time complexity?	O(n log n), mainly due to sorting.

4. N-Queens

Question	Short oral answer
What is N-Queens?	Place N queens on an N×N board so none attack each other.
What is backtracking?	Try a choice and undo it if it leads to failure.
What does isSafe() check?	Column, upper-left diagonal and upper-right diagonal.
Why check previous rows only?	Queens are placed row by row from top to bottom.
What does board[i][j]=1 mean?	A queen is placed in that cell.
What does board[i][j]=0 mean?	The cell is empty; it also undoes a placement.
What is the base case?	row == n; all queens have been placed.

5. Quick Sort

Question	Short oral answer
What is Quick Sort?	A divide-and-conquer sorting algorithm.
What is pivot?	The element used to partition the array.
What is partitioning?	Placing values on either side of the pivot based on comparison.

Question	Short oral answer
Deterministic Quick Sort here?	Always selects the last element as pivot.
Randomized Quick Sort?	Selects a random pivot from the current partition.
Average time complexity?	O(n log n).
Worst-case time complexity?	$O(n^2)$.
Why clone the array?	Both algorithms receive the same original input.
What does nanoTime() do?	Measures elapsed execution time at high resolution.

Most important differences to remember

Concept	Difference
Recursion vs Iteration	Recursion uses method calls; iteration uses loops.
Recursive vs Iterative Fibonacci	Recursive repeats calculations; iterative reuses previous two values.
Greedy vs Backtracking	Greedy makes a locally best choice; backtracking explores choices and reverses failed ones.
Huffman vs Fractional Knapsack	Huffman builds a frequency-based coding tree; Fractional Knapsack selects items by profit/weight ratio.
0/1 vs Fractional Knapsack	0/1 takes an item fully or not at all; fractional allows part of an item.
N-Queens vs Knapsack	N-Queens is a constraint/backtracking problem; Fractional Knapsack is a greedy optimization problem.
Deterministic vs Randomized Quick Sort	Deterministic uses a fixed pivot rule; randomized chooses a pivot randomly.
Quick Sort average vs worst case	Average $O(n \log n)$; worst $O(n^2)$.
Time vs Space Complexity	Time measures operation growth; space measures extra memory growth.

Quick memory line: Fibonacci = Recursion / Iteration; Huffman = Greedy + Priority Queue; Fractional Knapsack = Greedy ratio; N-Queens = Backtracking; Quick Sort = Divide and Conquer + Partition.

How to introduce each program in the external

Program	Short oral introduction
Fibonacci	I implemented Fibonacci using recursive and iterative methods and counted steps to compare the approaches.
Huffman Encoding	I counted character frequencies, built a Huffman tree using a priority queue, generated binary codes and encoded the text.
Fractional Knapsack	I calculated profit/weight ratios, sorted items in descending order and selected full or fractional items to maximize profit.
N-Queens	I used backtracking to place queens row by row while checking columns and diagonals for conflicts.
Quick Sort Analysis	I compared deterministic and randomized Quick Sort by counting comparisons and measuring execution time.

End of DAA Java viva notes